23CSE455 Design Patterns

Case Study Report

Team: A6
Case study: Ride-Sharing Application using Node.js/Express – *Design Pattern Analysis in a Node.js/Express Ride-Sharing Application*
Application: Sarathi backend (TypeScript, Express 4) – `https://github.com/AkhilMM2000/sarathi-backend`; faculty approval confirmed (reported by the team lead).
Date: 2026-10-11
Submission repository: 23CSE455_DP_CaseStudy

Students

Roll number	Student name	Batch
AM.SC.U4CSE23137	Dilip Kumar Reddy	CSE B
AM.SC.U4CSE23326	Sai Sameer	CSE D
AM.SC.U4CSE23134	M Kumar	CSE B
AM.SC.U4CSE23129	K Harish	CSE B
AM.SC.U4CSE23154	S Leela Shankar	TO FILL

1 Application and architecture

Original source: `https://github.com/AkhilMM2000/sarathi-backend`, default branch, baseline commit `c8766f4c88b9f36235d5f4399ef9b4cf20df3c8a` (2026-06-16, 185 commits), accessed 2026-10-11. **License:** the repository has no LICENSE file (`package.json` declares ISC); it is used here only for academic analysis.

Environment: TypeScript 5.9.3 on Node.js 22.18.0; Express 4.22; Mongoose 8.24 (MongoDB Atlas); tsyringe 4.10; Socket.IO 4.8; Upstash Redis; Stripe 18.5; Google Distance Matrix API; Zod 4.6. Versions are those resolved by `npm install` on 2026-10-11 because upstream ignores `package-lock.json`; with them `tsc -noEmit` reports 7 type errors, all in files we did not change (Stripe API-version literal, Express 5 typings, Upstash typings).

Scope: the seven areas of the case study – rider, driver (incl. reviews), trip, payment (Stripe, Stripe Connect, wallet), driver matching, location and notification – plus the framework code they use (DI container, middleware, error handling). Excluded as outside these areas: chat, vehicles, file upload, referral rewards, OTP/registration/OAuth/password reset, admin dashboards. Excluded from the copied code: `dist/` (compiled output), screenshots, a compiler log; demo credentials in the README were redacted.

Architecture: the code follows a four-layer clean architecture (Figure 1). A ride request enters `index.ts`, passes the route's middleware chain (`protectRoute` → `CheckBlockedUserOrDriver`), reaches `BookingController.bookDriver`, which validates the DTO with Zod and calls the `BookDriver` use case. `BookDriver` computes the fare, asks `IDriverRepository` for approved drivers within 30 km (MongoDB `$geoNear`), filters them to 20 km road distance through `GoogleDistanceService`, saves a PENDING booking through `IBookingRepository` and broadcasts `booking:new` through `INotificationService`, implemented by `SocketNotificationService` (Redis maps driver id to socket id). The first driver to call `AcceptBooking` wins an atomic PENDING→CONFIRMED update; `CronScheduler` expires requests nobody accepts. All collaborators are wired by the tsyringe container.

Run instructions: `README.md`. Running the server needs MongoDB, Redis, Stripe and Google credentials, which we did not have, so behaviour was verified through the real use-case classes with in-memory fakes: `cd Part2/code && npm install && npm test`. Baseline characterization run (unchanged `src`, test files added): 14/14 passed (`Part1/evidence/test_results.txt`).

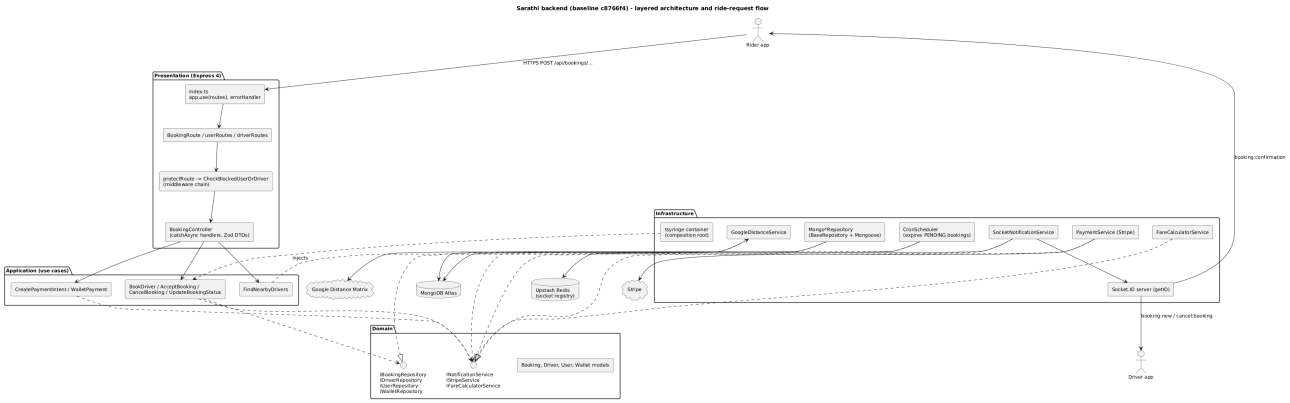


Figure 1: Layered architecture and ride-request flow (`Part1/diagrams/architecture.puml`).

2 Part 1 Pattern discovery

Method: (1) mapped the seven areas to files; (2) collected candidates from interfaces, DI registrations, naming and a first LLM pass (Section 5); (3) for every candidate traced implementers, injection points and call sites, and accepted it only when the roles of the pattern are present and interact; (4) recorded rejected and uncertain candidates with a reason; (5) wrote characterization tests for behaviour the changes would touch. A class name, decorator or LLM statement alone was never accepted.

2.1 Totals at the baseline

Confirmed pattern types: 10 – Layered Architecture, Dependency Injection, Singleton, Repository, Data Mapper, Adapter, Gateway, Observer, Chain of Responsibility, Decorator.
Confirmed pattern instances: 16.
Concrete participant classes: 11 (five Mongo repositories, `SocketNotificationService`, `PaymentService`, `StripeService`, `GoogleDistanceService`, `CheckBlockedUserOrDriver`, `BookingController`).
Unique participant symbols: 69 distinct classes, interfaces, functions and modules across all 16 records (shared symbols counted once; list in `Part1/evidence/participant_symbols.txt`).
Rejected and uncertain candidates: 6 rejected, 2 uncertain (Table 2); observation scope as above.

2.2 Pattern inventory

Pattern type	Instance names	Instances	Concrete cl.
Layered Architecture	Ride Booking Request Flow	1	0
Dependency Injection	Use-Case and Service Composition Root	1	0
Singleton	Shared Socket.IO Server Instance	1	0
Repository	Trip Booking Repository; Driver Repository with Geospatial Matching Query; Rider Repository; Rider Wallet Ledger Repository; Driver Review Repository	5	5
Data Mapper	Driver Review Persistence Mapping	1	0
Adapter	Socket.IO Ride Notification Adapter; Stripe Ride Payment Adapter; Stripe Connect Driver Payout Onboarding Adapter	3	3

Pattern type	Instance names	Instances	Concrete cl.
Gateway	Google Distance Matrix Location Gateway	1	1
Observer	Ride Event Notification	1	0
Chain of Responsibility	Authenticated Ride Request Pipeline	1	1
Decorator	Async Controller Error Forwarding	1	1

Table 2: Rejected and uncertain candidates (baseline).

Candidate	Verdict	Reason
Strategy	rejected	FareCalculatorService: only one implementation of IFareCalculatorService; ride-type behaviour is an if/else inside calculate (infrastructure/services/FareCalculatorService.ts:20-36). An interface used for DI is not a family of interchangeable algorithms.
State	rejected	BookingStatus and driver status are enums/strings; transitions are scattered if-checks (CancelBooking.ts:35-51, UpdateBookingstatus.ts:36-63, AdminChangeDriverStatus.ts:24-33). No state objects.
Command	rejected	Use cases expose execute(), but there is no invoker storing, queuing or undoing command objects; controllers call them directly. Classified as use-case interactors inside the Layered Architecture.
Template Method	rejected	BaseRepository is a generic layer supertype with complete methods; it defines no algorithm skeleton with abstract/hook steps (BaseRepository.ts:5-41).
Factory Method	rejected	protectRoute(allowedRoles) returns a configured closure (authMiddleware.ts:15-18); there is no creator hierarchy deciding which product class to instantiate.
Facade	rejected	Controllers delegate to one use case per action; they do not simplify a subsystem of several collaborating classes.
Adapter (Google Maps road distance)	uncertain	GoogleMapService implements IGoogleMapService (GoogleMapService.ts:8-70) and is registered (container.ts:99-101) but no class in the code injects IGoogleMapService, so no Client participant exists.
Adapter (Twilio SMS)	uncertain	TwilioSMSService implements SMSService (TwilioSMSService.ts:6-16) and is registered (container.ts:65) but nothing resolves 'SMSService'; dormant code.

2.3 One record for each pattern instance

Diagram for all records: Figure 2 (editable source `Part1/diagrams/baseline_patterns.puml`). Each row is also in `data/patterns.csv`.

Ride Booking Request Flow

Pattern type and category: Layered Architecture – Architectural.
Purpose and origin: Separates HTTP handling, business rules and persistence. The route only

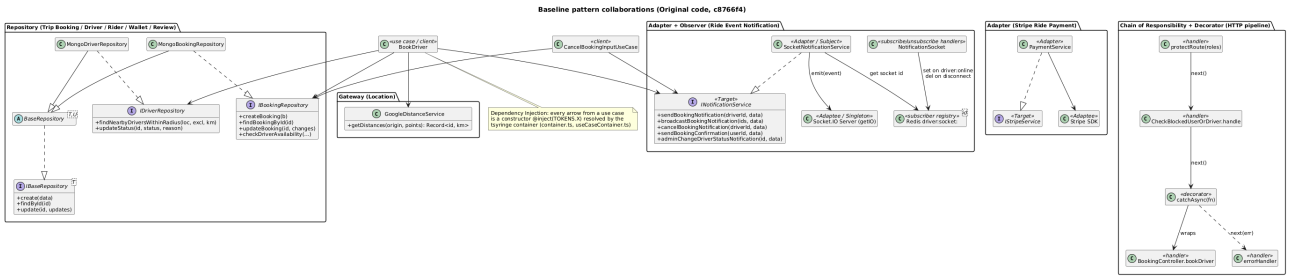


Figure 2: Main baseline collaborations (`Part1/diagrams/baseline_patterns.puml`).

wires middleware to BookingController.bookDriver; the controller validates the DTO and calls _bookDriverUseCase.execute; BookDriver depends only on domain interfaces (IBookingRepository, IDriverRepository, INotificationService) and never on Express; the Mongo implementations live in infrastructure. Dependencies point inward except the noted AcceptBooking violation (see rejected/uncertain notes).

Pattern role	Actual class, interface or function
PresentationLayer	BookingRoute router + BookingController.bookDriver
ApplicationLayer	BookDriver use case (IBookDriverUseCase)
DomainLayer	Booking entity + IBookingRepository/IDriverRepository ports
InfrastructureLayer	MongoBookingRepository/MongoDriverRepository + BookingModel

File and line ranges:

- `Part1/code/src/presentation/routes/BookingRoute.ts:10-17`
- `Part1/code/src/presentation/controllers/BookingController.ts:78-94`
- `Part1/code/src/application/use_cases/User/BookDriver.ts:27-142`
- `Part1/code/src/domain/models/Booking.ts:20-47`
- `Part1/code/src/domain/repositories/IBookingrepository.ts:23-62`
- `Part1/code/src/infrastructure/database/MongoBookingRepository.ts:12-24`

Verification: Confirmed (Dilip). Imports checked layer by layer; one violation (AcceptBooking imports BookingModel) recorded and removed in CH01.

Use-Case and Service Composition Root

Pattern type and category: Dependency Injection – Architectural (framework idiom - tsyringe IoC container).
Purpose and origin: Objects do not construct their collaborators. container.ts binds each symbol token to an implementation (e.g. TOKENS.IBOOKING_REPO -> MongoBookingRepository, TOKENS.NOTIFICATION_SERVICE -> SocketNotificationService); BookDriver declares @inject(TOKENS.X) constructor parameters typed by interfaces; routes obtain fully wired controllers with container.resolve. index.ts imports the container modules before any route so registrations exist at resolve time.

Pattern role	Actual class, interface or function
Injector/Container	tsyringe container
CompositionRoot	container.ts + useCaseContainer.ts + controllerContainer.ts
Tokens	TOKENS/USECASE_TOKENS symbols
Client	BookDriver (constructor @inject of 5 abstractions)
Services	MongoBookingRepository, MongoDriverRepository, FareCalculatorService, SocketNotificationService, GoogleDistanceService
Resolver	BookingRoute (container.resolve)

File and line ranges:

- `Part1/code/src/index.ts:1-6`
- `Part1/code/src/infrastructure/config/container.ts:44-101`
- `Part1/code/src/infrastructure/config/useCaseContainer.ts:168-199`
- `Part1/code/src/infrastructure/config/controllerContainer.ts:10-14`
- `Part1/code/src/constants/Tokens.ts:2-43`
- `Part1/code/src/application/use_cases/User/BookDriver.ts:27-35`
- `Part1/code/src/presentation/routes/BookingRoute.ts:11-12`

Verification: Confirmed. Every use case in scope is resolved from the container; tests construct the same classes with fakes, proving the constructor seams.

Shared Socket.IO Server Instance

Pattern type and category: Singleton – Creational.
Purpose and origin: Exactly one Socket.IO server must exist for the HTTP server. initializeSocket assigns the single module-level io once at start-up; every consumer obtains that same instance through getIO(), which throws if it is not initialised. SocketNotificationService uses a lazy getter so the instance is fetched only after initialisation. No class with a private constructor exists - the ES module cache provides the single instance.

Pattern role	Actual class, interface or function
Singleton	module-scoped variable io in socket.ts (no class
-	module plays the Singleton role)
Initializer	initializeSocket(server)
GlobalAccessPoint	getIO()
Clients	SocketNotificationService.io getter, NotificationSocket, index.ts

File and line ranges:

- `Part1/code/src/infrastructure/socket/socket.ts:9-16`
- `Part1/code/src/infrastructure/socket/socket.ts:181-184`
- `Part1/code/src/infrastructure/services/SocketNotification.ts:8-10`
- `Part1/code/src/infrastructure/socket/SocketNotification.ts:13-14`
- `Part1/code/src/index.ts:55-60`

Verification: Confirmed as a module-level Singleton; alternative 'registerSingleton = Singleton' was folded into DI (container lifetime, not a pattern collaboration).

Trip Booking Repository

Pattern type and category: Repository – Enterprise (PoEAA) / Domain-Driven Design.
Purpose and origin: Trip use cases see a collection-like domain interface (createBooking, findBookingById, updateBooking, checkDriverAvailability) instead of Mongoose. MongoBookingRepository

extends BaseRepository and maps calls onto BookingModel queries/aggregations; BookDriver and CancelBooking receive it through @inject(TOKENS.IBOOKING_REPO).

Pattern role	Actual class, interface or function
Repository	IBookingRepository
LayerSupertype	IBaseRepository + BaseRepository<T,U>
ConcreteRepository	MongoBookingRepository
Entity	Booking
PersistenceModel	BookingModel
Clients	BookDriver, CancelBookingInputUseCase, UpdateBookingStatus, AttachPaymentIntentIdToBooking

File and line ranges:

- `Part1/code/src/domain/repositories/IBookingrepository.ts:23-62`
- `Part1/code/src/domain/repositories/IBaseRepository.ts:1-8`
- `Part1/code/src/infrastructure/database/BaseRepository.ts:5-41`
- `Part1/code/src/infrastructure/database/MongoBookingRepository.ts:12-516`
- `Part1/code/src/domain/models/Booking.ts:20-47`
- `Part1/code/src/infrastructure/database/modals/Bookingschema.ts:4-54`
- `Part1/code/src/application/use_cases/User/BookDriver.ts:30,46,118`
- `Part1/code/src/application/use_cases/User/CancelBooking.ts:22-23,29,48-51`

Verification: Confirmed; characterization tests run BookDriver/CancelBooking against an in-memory implementation of the same interface.

Driver Repository with Geospatial Matching Query

Pattern type and category: Repository – Enterprise (PoEAA) / Domain-Driven Design.
Purpose and origin: Driver matching asks the repository for drivers within a radius; MongoDriverRepository hides the $geoNear aggregation (approved, unblocked, activePayment drivers) behind findNearbyDriversWithinRadius. Status changes go through updateStatus. Clients depend on the interface via @inject(TOKENS.IDRIVER_REPO).

Pattern role	Actual class, interface or function
Repository	IDriverRepository
LayerSupertype	BaseRepository<T,U>
ConcreteRepository	MongoDriverRepository
Entity	Driver
PersistenceModel	DriverModel (2dsphere index)
Clients	BookDriver.findNearbyDriversWithinRadius, FindNearbyDrivers.findActiveDrivers, AdminChangeDriverStatus.updateStatus, CreatePaymentIntent.findDriverById

File and line ranges:

- `Part1/code/src/domain/repositories/IDriverepository.ts:4-27`
- `Part1/code/src/infrastructure/database/MongodriverRepository.ts:13-289`
- `Part1/code/src/infrastructure/database/MongodriverRepository.ts:243-287`
- `Part1/code/src/domain/models/Driver.ts:3-37`
- `Part1/code/src/infrastructure/database/modals/Driverschema.ts:4-74`
- `Part1/code/src/application/use_cases/User/BookDriver.ts:64-68`
- `Part1/code/src/application/use_cases/Admin/AdminChangeDriverStatus.ts:33`

Verification: Confirmed; the $geoNear filter (approved, unblocked, activePayment) is what lets CH01

exclude suspended drivers without a query change.

Rider Repository

Pattern type and category: Repository – Enterprise (PoEAA) / Domain-Driven Design.
Purpose and origin: Rider data (location used for matching, block flag used by middleware) is read through IUserRepository.getUserById; the Mongo implementation extends BaseRepository. Clients never touch UserModel directly.

Pattern role	Actual class, interface or function
Repository	IUserRepository
LayerSupertype	BaseRepository<T,U>
ConcreteRepository	MongoUserRepository
Entity	User
PersistenceModel	UserModel (userschema)
Clients	FindNearbyDrivers (rider location), CheckBlockedUserOrDriver, AttachPaymentIntentIdToBooking

File and line ranges:

- `Part1/code/src/domain/repositories/IUserepository.ts:3-12`
- `Part1/code/src/infrastructure/database/MongoUserRepository.ts:13-150`
- `Part1/code/src/domain/models/User.ts:3-26`
- `Part1/code/src/infrastructure/database/modals/userschema.ts:4-45`
- `Part1/code/src/application/use_cases/User/FindNearbyDrivers.ts:17,27`
- `Part1/code/src/middleware/checkBlocked.ts:13,38`

Verification: Confirmed (clients in scope only; auth clients excluded by scope).

Rider Wallet Ledger Repository

Pattern type and category: Repository – Enterprise (PoEAA) / Domain-Driven Design.
Purpose and origin: Ride payment use cases credit/debit the rider wallet through creditAmount/debitAmount on the interface; MongoWalletRepository keeps the transaction list consistent in MongoDB.

Pattern role	Actual class, interface or function
Repository	IWalletRepository
LayerSupertype	BaseRepository<T,U>
ConcreteRepository	MongoWalletRepository
Entity	Wallet/WalletTransaction
PersistenceModel	WalletModel
Clients	AttachPaymentIntentIdToBooking (debit), WalletPayment.WalletRidePayment (debit), WalletService

File and line ranges:

- `Part1/code/src/domain/repositories/IWalletRepository.ts:3-10`
- `Part1/code/src/infrastructure/database/WalletRepository.ts:13-134`
- `Part1/code/src/domain/models/Wallet.ts:3-18`
- `Part1/code/src/infrastructure/database/modals/Walletschema.ts:5-35`
- `Part1/code/src/application/use_cases/User/AttachPaymentIntentIdToBooking.ts:20,35-42`
- `Part1/code/src/application/use_cases/User/WalletRidePayment.ts:16,53-57`

Verification: Confirmed.

Driver Review Repository

Pattern type and category: Repository – Enterprise (PoEAA) / Domain-Driven Design.
Purpose and origin: Driver ratings are stored and queried through IDriverReviewRepository (createReview, hasUserAlreadyReviewed, getReviewsByDriverId); the use cases rely only on that interface.

Pattern role	Actual class, interface or function
Repository	IDriverReviewRepository
LayerSupertype	BaseRepository<T,U>
ConcreteRepository	MongoDriverReviewRepository
Entity	DriverReview
PersistenceModel	ReviewModel
Clients	SubmitDriverReview, GetDriverReviews

File and line ranges:

- `Part1/code/src/domain/repositories/IDriverReviewRepository.ts:8-12`
- `Part1/code/src/infrastructure/database/MongoDriverReviewRepository.ts:11-53`
- `Part1/code/src/domain/models/DriverReview.ts:1-14`
- `Part1/code/src/application/use_cases/User/SubmitRating.ts:12-17,22,27`
- `Part1/code/src/application/use_cases/Driver/DriverReview.ts:7-14`

Verification: Confirmed.

Driver Review Persistence Mapping

Pattern type and category: Data Mapper – Enterprise (PoEAA).
Purpose and origin: Keeps the domain object free of Mongoose types: toPersistence converts string ids to ObjectId before save and toDomain converts the stored document (and populated user) back to a plain DriverReview. The repository calls toPersistence before create and maps every query result with toDomain.

Pattern role	Actual class, interface or function
Mapper	DriverReviewMapper.toDomain / toPersistence (functions, no class)
DomainObject	DriverReview
PersistenceRecord	ReviewDocument
Client	MongoDriverReviewRepository

File and line ranges:

- `Part1/code/src/infrastructure/database/mappers/DriverReviewMapper.ts:6-28`
- `Part1/code/src/domain/models/DriverReview.ts:1-14`
- `Part1/code/src/infrastructure/database/modals/DriverReviewschema.ts:4-22`
- `Part1/code/src/infrastructure/database/MongoDriverReviewRepository.ts:17-19,45`

Verification: Confirmed; the only explicit domain/persistence mapper in scope.

Socket.IO Ride Notification Adapter

Pattern type and category: Adapter – Structural.
Purpose and origin: Use cases speak a domain vocabulary (sendBookingNotification, cancelBookingNotification, paymentNotification...). SocketNotificationService translates each call into a Redis lookup of the recipient socket id followed by io.to(socketId).emit('<event>', data), so use cases never import Socket.IO or Redis.

Pattern role	Actual class, interface or function
Target	INotificationService
Adapter	SocketNotificationService
Adaptee	Socket.IO Server (getIO) + Upstash Redis socket registry (redis.get driver:socket:<id>)
Clients	BookDriver, AcceptBooking, CancelBookingInputUseCase, UpdateBookingStatus, AdminChangeDriverStatus, AttachPaymentIntentIdToBooking, CronScheduler

File and line ranges:

- `Part1/code/src/application/services/NotificationService.ts:2-16`
- `Part1/code/src/infrastructure/services/SocketNotification.ts:6-72`
- `Part1/code/src/infrastructure/socket/socket.ts:181-184`
- `Part1/code/src/config/redisconfig.ts:6-9`
- `Part1/code/src/infrastructure/config/container.ts:88-90`
- `Part1/code/src/application/use_cases/User/BookDriver.ts:120-138`
- `Part1/code/src/application/use_cases/User/CancelBooking.ts:52-54`

Verification: Confirmed; distinguished from the Observer instance: the adapter is the translation of a domain interface to Socket.IO, the Observer is the subscribe/notify collaboration.

Stripe Ride Payment Adapter

Pattern type and category: Adapter – Structural.
Purpose and origin: The domain-level interface expresses ride payments (createPaymentIntent with platformFee, transferToDriverFromWallet). PaymentService converts those calls into Stripe API calls (paymentIntents.create with application_fee_amount and transfer_data, transfers.create in paisa).

Pattern role	Actual class, interface or function
Target	IStripeService
Adapter	PaymentService
Adaptee	Stripe SDK client (paymentIntents, transfers, balance)
Clients	CreatePaymentIntent, AttachPaymentIntentIdToBooking, WalletPayment

File and line ranges:

- `Part1/code/src/domain/services/IStripeService.ts:3-22`
- `Part1/code/src/infrastructure/services/StripeService.ts:8-91`
- `Part1/code/src/infrastructure/config/container.ts:75-77`
- `Part1/code/src/application/use_cases/User/CreatePaymentIntent.ts:11-39`
- `Part1/code/src/application/use_cases/User/AttachPaymentIntentIdToBooking.ts:22-23,51-56`
- `Part1/code/src/application/use_cases/User/WalletRidePayment.ts:19-20,47-51`

Verification: Confirmed.

Stripe Connect Driver Payout Onboarding Adapter

Pattern type and category: Adapter – Structural.
Purpose and origin: Driver payout onboarding needs 'create account', 'create onboarding link' and 'retrieve account'. StripeService implements those three operations by calling stripe.accounts.create (express), stripe.accountLinks.create and stripe.accounts.retrieve.

Pattern role	Actual class, interface or function
Target	IStripeAccountService
Adapter	StripeService (Accountservice.ts)
Adaptee	module-level Stripe SDK client (accounts, accountLinks)
Clients	OnboardDriverUseCase, VerifyDriverPaymentAccount

File and line ranges:

- `Part1/code/src/application/services/Accountservice.ts:3-7`
- `Part1/code/src/infrastructure/services/Accountservice.ts:7-60`
- `Part1/code/src/infrastructure/config/container.ts:72-74`
- `Part1/code/src/application/use_cases/Driver/DriverOnboarding.ts:7-20`
- `Part1/code/src/application/use_cases/Driver/VerifyAccountStatus.ts:11-23`

Verification: Confirmed.

Google Distance Matrix Location Gateway

Pattern type and category: Gateway – Enterprise (PoEAA).
Purpose and origin: All road-distance lookups go through one object that builds the origins/destinations query, calls the API and returns a Record<driverOrBookingId, km>. Callers never see HTTP details. It is a Gateway rather than an Adapter because there is no separate target interface; clients inject the concrete class.

Pattern role	Actual class, interface or function
Gateway	GoogleDistanceService.getDistances (concrete class, no interface)
ExternalResource	Google Distance Matrix REST API via axios
Clients	BookDriver (road-distance filter <= 20 km), FindNearbyDrivers (sort by distance), NotificationSocket driver:online catch-up

File and line ranges:

- `Part1/code/src/application/services/GoogleDistanceService.ts:4-45`
- `Part1/code/src/infrastructure/config/container.ts:64`
- `Part1/code/src/application/use_cases/User/BookDriver.ts:71-93`
- `Part1/code/src/application/use_cases/User/FindNearbyDrivers.ts:67-72`
- `Part1/code/src/infrastructure/socket/SocketNotification.ts:45-62`

Verification: Confirmed as Gateway. Rejected alternative: Adapter (no target interface). The interface-based GoogleMapService is uncertain because nothing injects it.

Ride Event Notification

Pattern type and category: Observer – Behavioral.
Purpose and origin: Riders and drivers register interest when their client emits driver:online/user:online (socket id stored in Redis) and are removed on disconnect. When a trip event happens (new booking, acceptance, cancellation, payment) the subject pushes it to every registered recipient with emit; broadcastBookingNotification loops over nearby driver ids and silently skips drivers with no socket. Publishers do not know how many observers exist.

Pattern role	Actual class, interface or function
Subject	SocketNotificationService over the Socket.IO server (subject role combined, not separately declared)
SubscriberRegistry	Redis keys driver:socket:<id>/user:socket:<id>
Subscribe	NotificationSocket driver:online/user:online handlers
Unsubscribe	disconnect handler
Notify	io.to(socketId).emit(event)
ConcreteObservers	connected rider/driver Socket.IO clients (outside this repository)

File and line ranges:

- `Part1/code/src/infrastructure/socket/SocketNotification.ts:16-20`
- `Part1/code/src/infrastructure/socket/SocketNotification.ts:85-106`
- `Part1/code/src/infrastructure/services/SocketNotification.ts:11-26`
- `Part1/code/src/infrastructure/services/SocketNotification.ts:27-70`
- `Part1/code/src/application/use_cases/Driver/AcceptBooking.ts:37-50`

Verification: Confirmed with roles combined: the subject is not a separate class and the concrete observers are client sockets outside the repository.

Authenticated Ride Request Pipeline

Pattern type and category: Chain of Responsibility – Behavioral.
Purpose and origin: Each handler either ends the request (401 no token, 403 wrong role, 403 blocked account) or calls next() to pass it on; errors thrown later travel to errorHandler, which picks the response by error type. The order is assembled per route, e.g. protectRoute(['user']) -> checkBlocked -> bookingController.bookDriver.

Pattern role	Actual class, interface or function
Handler	Express middleware signature (req,res,next) (framework, not declared)
ConcreteHandlers	protectRoute(roles) closure, CheckBlockedUserOrDriver.handle, catchAsync-wrapped BookingController handler, errorHandler
Client/ChainBuilder	BookingRoute, userRoutes and driverRoutes route definitions + app.use in index.ts

File and line ranges:

- `Part1/code/src/middleware/authMiddleware.ts:15-60`
- `Part1/code/src/middleware/checkBlocked.ts:10-63`
- `Part1/code/src/middleware/errorHandler.ts:12-56`
- `Part1/code/src/presentation/routes/BookingRoute.ts:14-16`
- `Part1/code/src/presentation/routes/userRoutes.ts:102-106`
- `Part1/code/src/index.ts:34-51`

Verification: Confirmed; each handler can stop the request or call next().

Async Controller Error Forwarding

Pattern type and category: Decorator – Structural (function-wrapper form).
Purpose and origin: catchAsync returns a RequestHandler with the same (req,res,next) call shape as the wrapped controller function and adds one responsibility: a rejected promise is forwarded to next(err). Every BookingController action is declared as catchAsync(async ...), so error forwarding is added without modifying the handlers.

Pattern role	Actual class, interface or function
Component	Express RequestHandler interface
Decorator	catchAsync(fn) (higher-order function)
ConcreteComponents	BookingController async handlers (bookDriver, cancelBooking, updateStatus...)
Client	route definitions that register the wrapped handlers

File and line ranges:

- `Part1/code/src/infrastructure/utils/catchAsync.ts:7-13`
- `Part1/code/src/presentation/controllers/BookingController.ts:78-94`
- `Part1/code/src/presentation/controllers/BookingController.ts:165-181`
- `Part1/code/src/presentation/controllers/BookingController.ts:193-204`

Verification: Confirmed in function-wrapper form; same call shape, one added responsibility.

3 Part 2 Evolution and refactoring

The four requirements were delivered as three changes on branch `a6-evolution`; each was implemented, tested and committed separately. Details, patches and logs: `Part2/changes/CH0x/`.

CH01 – New driver status (SUSPENDED)

Change and acceptance criteria: an admin can suspend an approved driver with a reason and reinstate them later. Only pending→approved/rejected, approved→suspended, suspended→approved/rejected and rejected→pending are legal (409 otherwise, nothing saved or sent). A suspended driver is not matched, cannot accept or be requested directly, receives no catch-up requests and cannot log in.
Before and after: `168ef00` → `f635355`; 15 source files (`Part2/changes/CH01/diffstat.txt`).
Implementation and tests: State pattern in `domain/driver/DriverAccountState.ts`; `AdminChangeDriverStatus`, `AcceptBooking`, `BookDriver` and the socket catch-up ask the state. `npm test`: 22/22 passed (`Part2/changes/CH01/test_results.txt`).
Pattern effects: Driver Account Lifecycle newly introduced; driver repository modified; trip repository extended; Observer and Layered Architecture modified; DI extended (Table 3).
Refactoring: *smell* – the application-layer `AcceptBooking` ran a Mongoose query itself; *technique* – Move Method into `IBookingRepository.acceptPendingBooking`; *evidence* – a test checks the same CONFIRMED result, notification order and first-driver-wins behaviour.
Before/after metrics: `AdminChangeDriverStatus` max CC 5→3; `AcceptBooking` NLOC 42→58, CC 4→6 (new state check); new state module 135 NLOC.

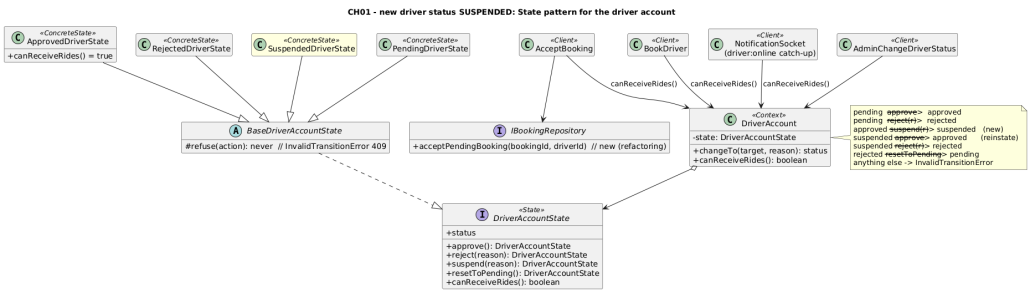


Figure 3: CH01 State.

CH02 – Ride cancellation

Change and acceptance criteria: riders and assigned drivers can cancel only their own PENDING or CONFIRMED rides; finished rides return 409. Rider fee is 0 while PENDING or before 21:00 IST the day before, otherwise max(Rs 50, 10% of the fare); driver cancellations are free and notify the rider; unaccepted broadcasts dismiss the drivers' popups.
Before and after: `f635355` → `28a83c8`; 13 source files.
Implementation and tests: State (`TripLifecycle`) used by cancel, status update and expiry; Strategy (`CancellationPolicy`) for the fee. `npm test`: 33/33 passed.
Pattern effects: two new instances; notification adapter, Observer, middleware chain, `catchAsync` decorator and trip repository extended.
Refactoring: *smells* – a forced `status` parameter, cutoff arithmetic inside the use case, no status guard (the baseline test *a COMPLETED trip can still be cancelled* passed), no ownership check; *techniques* – Replace Conditional with State, Extract Class; *evidence* – the baseline cancellation tests keep their expectations.
Before/after metrics: `CancelBooking` NLOC 48→48 but max CC 5→9 (ownership and recipient branches); new `TripState.ts` 114 NLOC (max CC 4) and `CancellationPolicy.ts` 37 NLOC.

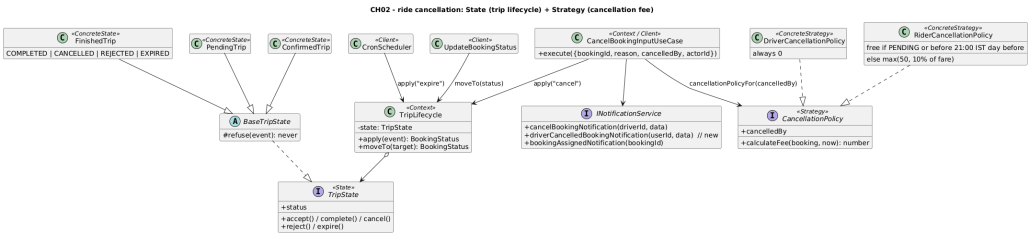


Figure 4: CH02 State + Strategy.

CH03 – Multiple ride types and dynamic pricing

Change and acceptance criteria: new HOURLY (Rs 200 per started hour, min 4 h) and OUTSTATION (Rs 12/km + Rs 300/day) ride types; +25% in peak hours (08:00–10:59, 17:00–20:59 IST) and +20% when fewer than 3 drivers are in range; existing fares unchanged off-peak; final fare uses each type's extra-km rate.
Before and after: `28a83c8` → `a12c878`; 7 source files.
Implementation and tests: Strategy per ride type in `domain/pricing/FareStrategy.ts`, Decorators in `DynamicPricing.ts`, `FareCalculatorService` as context; `BookDriver` prices after matching. `npm test`: 41/41 passed; the six baseline fare tests pass unchanged.
Pattern effects: Ride Fare Pricing (Strategy) and Dynamic Fare Pricing (Decorator) newly introduced; DI extended.
Refactoring: *smells* – switch on `bookingType`, magic number `ratePerKm = 10` duplicated in `UpdateBookingStatus`, plain `Error` turning into HTTP 500; *techniques* – Replace Conditional with Polymorphism, Replace Magic Number; *evidence* – unchanged characterization tests.
Before/after metrics: `FareCalculatorService` NLOC 32→30, max CC 6→2; `BookDriver` max CC 15→18 (driver-state check and supply argument).

Records of the newly introduced instances

Driver Account Lifecycle

Pattern type and category: State – Behavioral.
Purpose and origin: Each status object answers which admin actions are legal and whether the driver may take rides. DriverAccount.changeTo maps the requested status to approve/reject/suspend/resetToPending on the current state, which returns the next state or throws

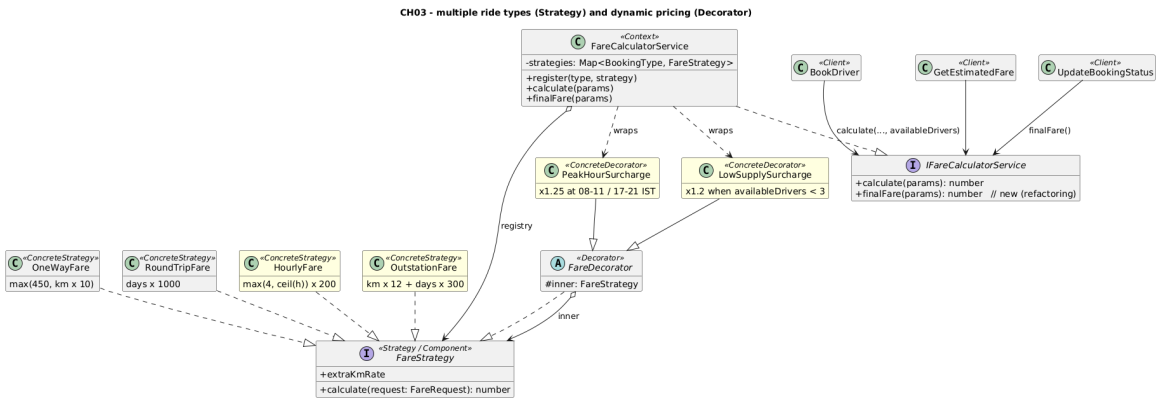


Figure 5: CH03 Strategy + Decorator.

InvalidTransitionError (409). Adding SUSPENDED needed one new class plus suspend() on ApprovedDriverState; tests: tests/driverStatus.test.ts (9 cases).

Pattern role	Actual class, interface or function
State	DriverAccountState (+ BaseDriverAccountState default refusals)
ConcreteStates	PendingDriverState, ApprovedDriverState, RejectedDriverState, SuspendedDriverState
Context	DriverAccount
Clients	AdminChangeDriverStatus, AcceptBooking, BookDriver, NotificationSocket driver:online handler

File and line ranges:

- `Part2/code/src/domain/driver/DriverAccountState.ts:13-50`
- `Part2/code/src/domain/driver/DriverAccountState.ts:52-92`
- `Part2/code/src/domain/driver/DriverAccountState.ts:110-144`
- `Part2/code/src/application/use_cases/Admin/AdminChangeDriverStatus.ts:31`
- `Part2/code/src/application/use_cases/Driver/AcceptBooking.ts:29`
- `Part2/code/src/application/use_cases/User/BookDriver.ts:48`

Verification: Confirmed by unit tests listed in the change README.

Trip Lifecycle and Cancellation

Pattern type and category: State – Behavioral.
Purpose and origin: Trip transitions are decided by the current state object: PendingTrip accepts accept/cancel/reject/expire, ConfirmedTrip accepts complete/cancel/reject, FinishedTrip refuses everything. CancelBooking calls apply('cancel'), UpdateBookingStatus calls moveTo(status) and CronScheduler calls apply('expire'). Fixes the baseline defect where a COMPLETED trip could be cancelled; tests T1-T8 in tests/cancellation.test.ts.

Pattern role	Actual class, interface or function
State	TripState (+ BaseTripState default refusals)
ConcreteStates	PendingTrip, ConfirmedTrip, FinishedTrip (COMPLETED/CANCELLED/REJECTED/EXPIRED)
Context	TripLifecycle
Clients	CancelBookingInputUseCase, UpdateBookingStatus, CronScheduler

File and line ranges:

- `Part2/code/src/domain/booking/TripState.ts:10-75`

- Part2/code/src/domain/booking/TripState.ts:97-120
- Part2/code/src/application/use_cases/User/CancelBooking.ts:35
- Part2/code/src/application/use_cases/Driver/UpdateBookingstatus.ts:44
- Part2/code/src/infrastructure/services/CronScheduler.ts:30

Verification: Confirmed by unit tests listed in the change README.

Cancellation Fee Policy

Pattern type and category: Strategy – Behavioral.
Purpose and origin: The fee rule varies by who cancels. The rider policy is free while PENDING or before 21:00 IST the day before the ride, else max(Rs 50, 10% of the estimated fare); the driver policy is always 0. The use case asks cancellationPolicyFor(cancelledBy) and calls calculateFee without knowing which rule runs.

Pattern role	Actual class, interface or function
Strategy	CancellationPolicy
ConcreteStrategies	RiderCancellationPolicy, DriverCancellationPolicy
Selector	cancellationPolicyFor(cancelledBy)
Context	CancelBookingInputUseCase

File and line ranges:

- Part2/code/src/domain/booking/CancellationPolicy.ts:6-9
- Part2/code/src/domain/booking/CancellationPolicy.ts:26-41
- Part2/code/src/domain/booking/CancellationPolicy.ts:43-50
- Part2/code/src/application/use_cases/User/CancelBooking.ts:36

Verification: Confirmed by unit tests listed in the change README.

Ride Fare Pricing

Pattern type and category: Strategy – Behavioral.
Purpose and origin: One class per ride type computes the base fare and exposes its extra-km rate. FareCalculatorService looks the strategy up by bookingType; a new ride type is a new class plus one register() call. The 6 baseline characterization tests in tests/fare.test.ts still pass unchanged (behaviour preserved for ONE_WAY/ROUND_TRIP).

Pattern role	Actual class, interface or function
Strategy	FareStrategy
ConcreteStrategies	OneWayFare, RoundTripFare, HourlyFare, OutstationFare
Context	FareCalculatorService (registry Map<BookingType, FareStrategy>, register())
Clients	BookDriver, GetEstimatedFare, UpdateBookingStatus (finalFare)

File and line ranges:

- Part2/code/src/domain/pricing/FareStrategy.ts:15-68
- Part2/code/src/infrastructure/services/FareCalculatorService.ts:10-35
- Part2/code/src/application/services/FareCalculatorService.ts:4-7
- Part2/code/src/application/use_cases/User/BookDriver.ts:100-106
- Part2/code/src/application/use_cases/Driver/UpdateBookingstatus.ts:60-64

Verification: Confirmed by unit tests listed in the change README.

Dynamic Fare Pricing

Pattern type and category: Decorator – Structural.
Purpose and origin: Pricing rules wrap any ride-type strategy with the same interface: PeakHourSurcharge multiplies by 1.25 for starts 08:00-10:59 and 17:00-20:59 IST, LowSupplySurcharge by 1.2 when fewer than 3 drivers are in range; they stack (60 km evening ride with 1 driver: 600 -> 750 -> 900). tests/pricing.test.ts.

Pattern role	Actual class, interface or function
Component	FareStrategy
ConcreteComponents	OneWayFare/RoundTripFare/HourlyFare/OutstationFare
Decorator	FareDecorator (holds inner FareStrategy)
ConcreteDecorators	PeakHourSurcharge, LowSupplySurcharge
Client	FareCalculatorService.calculate (wraps the selected strategy)

File and line ranges:

- `Part2/code/src/domain/pricing/DynamicPricing.ts:7-40`
- `Part2/code/src/infrastructure/services/FareCalculatorService.ts:22-25`
- `Part2/code/src/domain/pricing/FareStrategy.ts:15-19`

Verification: Confirmed by unit tests listed in the change README.

3.1 Updated pattern counts

Snapshot	Types	Instances	Concrete classes	Rejected / uncertain (baseline scope)
Original code	10	16	11	6 rejected, 2 uncertain
After CH01	11	17	15	6 rejected, 2 uncertain
After CH02	12	19	20	6 rejected, 2 uncertain
After CH03	12	21	26	6 rejected, 2 uncertain

No instance was removed or replaced; continuing collaborations keep their instance names. The Ride Fare Pricing Strategy replaces the conditional that was the rejected Strategy candidate of the baseline, and the two State instances replace the status conditionals that were the rejected State candidate.

Table 3: Instances whose status changed (all other rows: preserved; full table `data/patterns_evolution.csv`).

Snapshot	Instance	Status	Change
After CH01	Ride Booking Request Flow (Layered Architecture)	modified	AcceptBooking no longer imports the Mongoose BookingModel (application -> infrastructure dependency removed); new domain module domain/driver holds the driver-account states.
After CH01	Use-Case and Service Composition Root (Dependency Injection)	extended	AcceptBooking gains @inject(TOKENS.IDRIVER_REPO); no new registrations needed.
After CH01	Trip Booking Repository (Repository)	extended	New repository operation acceptPendingBooking (atomic PENDING->CONFIRMED) moved out of AcceptBooking; AcceptBooking becomes a new client.

Snapshot	Instance	Status	Change
After CH01	Driver Repository with Geospatial Matching Query (Repository)	modified	updateStatus accepts DriverAccountStatus (adds suspended) and stores the suspension reason; schema enum extended. The geo query is unchanged and already excludes non-approved (suspended) drivers.
After CH01	Ride Event Notification (Observer)	modified	The driver:online subscription handler now skips the catch-up push for drivers whose account state cannot receive rides (pending/rejected/suspended); the admin status notification is sent only after a legal transition is saved.
After CH01	Driver Account Lifecycle (State)	newly_ introduced	Requirement CH01 (new driver status). Replaces the any-to-any status update validated by an array includes() in AdminChangeDriverStatus (rejected State candidate in the baseline).
After CH02	Trip Booking Repository (Repository)	extended	Booking entity and schema gain cancelledBy and cancellationFee, persisted through updateBooking by CancelBookingInputUseCase.
After CH02	Socket.IO Ride Notification Adapter (Adapter)	extended	Target INotificationService gains driverCancelledBookingNotification; the adapter maps it to emit('booking:cancelledByDriver') on the rider socket.
After CH02	Ride Event Notification (Observer)	extended	New published event booking:cancelledByDriver to riders; cancelling an unaccepted broadcast publishes booking:assigned so drivers dismiss the request.
After CH02	Authenticated Ride Request Pipeline (Chain of Responsibility)	extended	New chain for drivers: protectRoute(['driver']) -> CheckBlockedUserOrDriver.handle -> BookingCon- troller.driverCancelBooking.
After CH02	Async Controller Error Forwarding (Decorator)	extended	New ConcreteComponent driverCancelBooking wrapped by catchAsync; cancelBooking now reads the authenticated user id.
After CH02	Trip Lifecycle and Cancellation (State)	newly_ introduced	Requirement CH02 (ride cancellation). Replaces scattered status if-checks in CancelBooking and UpdateBookingStatus.
After CH02	Cancellation Fee Policy (Strategy)	newly_ introduced	Requirement CH02. The baseline cutoff rule (refuse after 21:00 IST) moved from CancelBooking into RiderCancellationPolicy and became a late fee.

Snapshot	Instance	Status	Change
After CH03	Use-Case and Service Composition Root (Dependency Injection)	extended	UpdateBookingStatus gains @inject(TOKENS.IFARE_CALCULATE_SERVICE).
After CH03	Ride Fare Pricing (Strategy)	newly_ introduced	Requirement CH03 (multiple ride types). Replaces the if/else in FareCalculatorService.calculate (the rejected Strategy candidate of the baseline) and the duplicated ratePerKm = 10 in UpdateBookingStatus.
After CH03	Dynamic Fare Pricing (Decorator)	newly_ introduced	Requirement CH03 (dynamic pricing). BookDriver now prices after matching so availableDrivers is known.

Same-scope metrics of the edited and new files after CH03 (`tools/ts_metrics.js`; NLOC = non-blank, non-comment lines):

File	NLOC	max CC
`application/use_cases/Admin/AdminChangeDriverStatus.ts`	32	3
`application/use_cases/Driver/AcceptBooking.ts`	58	6
`application/use_cases/User/CancelBooking.ts`	48	9
`application/use_cases/Driver/UpdateBookingstatus.ts`	64	12
`infrastructure/services/FareCalculatorService.ts`	30	2
`application/use_cases/User/BookDriver.ts`	136	18
`infrastructure/services/CronScheduler.ts`	45	5
`domain/driver/DriverAccountState.ts`	135	6
`domain/booking/TripState.ts`	114	4
`domain/booking/CancellationPolicy.ts`	37	4
`domain/pricing/FareStrategy.ts`	52	3
`domain/pricing/DynamicPricing.ts`	28	5

Whole `src`: 8,232 NLOC / 399 functions before, 8,680 NLOC / 460 functions after; maximum function CC 17 → 18.

Interpretation: the new states and strategies made each requirement an additive change: SUSPENDED is one new class and one method, a new ride type or pricing rule is one class plus a registration, and illegal transitions are refused in one place. The cost is about 450 more NLOC and more, smaller functions; some use cases (`CancelBooking`, `BookDriver`) gained complexity because they now enforce ownership and supply rules. More or fewer LOC alone does not prove a better design; the evidence for the improvement is that two baseline defects are now caught by tests and that the old fare behaviour is preserved by unchanged tests. Remaining limitations: `UpdateBookingStatus` still does not check that the caller is the booking's driver; the pre-booking estimate cannot include the low-supply surcharge.

4 Part 3 Cross-language comparison

Selected instances: Ride Fare Pricing (Strategy, CH03), Trip Lifecycle and Cancellation (State, CH02), Ride Event Notification (Observer, baseline). These are the patterns the requirements changed

most; the infrastructure patterns (Repository, Adapter, Gateway, Data Mapper, DI) depend on MongoDB, Stripe, Google or tsyringe and have no self-contained four-language equivalent with identical behaviour.

Common specifications: `Part3/test-cases/strategy_spec.md` (S1–S8: fares for every ride type, invalid input, unknown type), `state_spec.md` (T1–T8: event sequences, final state and number of refused events), `observer_spec.md` (O1–O6: broadcast, unsubscribe, reconnect replacement, ordering). S1–S8 and T1–T8 are also asserted against the TypeScript code.

Implementations: `Part3/<pattern>/{java,python,javascript,cpp}/` – Java 25, Python 3.13.7, Node.js 22.18.0, g++ 13.5.0 (`-std=c++20 -O2`, Docker `gcc:13`); commands in each folder's README and `tools/run_part3.py`.

Actual results and static/dynamic comparison (`data/cross_language.csv`; LOC = lizard NLOC of the production file; CC = maximum function cyclomatic complexity; time = median of 5 complete test processes after 1 warm-up):

Pattern	Language	LOC	max CC	Tests	Median ms
Strategy	Java	57	3	8/8 passed	102.7
Strategy	Python	51	3	8/8 passed	59.8
Strategy	JavaScript	47	3	8/8 passed	49.4
Strategy	C++	69	3	8/8 passed	2.1
State	Java	58	6	8/8 passed	98.3
State	Python	52	2	8/8 passed	31.0
State	JavaScript	41	2	8/8 passed	45.3
State	C++	57	6	8/8 passed	1.9
Observer	Java	33	3	6/6 passed	110.8
Observer	Python	31	3	6/6 passed	63.5
Observer	JavaScript	24	2	6/6 passed	46.3
Observer	C++	40	3	6/6 passed	2.3

All 12 implementations passed every fixture. Times include process start-up and are dominated by it (JVM about 100 ms, Node about 47 ms, CPython 31–64 ms, native C++ about 2 ms measured inside the container); they are not throughput benchmarks.

Comparison:

Snippet	Observed language-specific structure
Strategy / Java	Nominal FareStrategy interface plus nested static classes and a Trip record; the compiler rejects a strategy that lacks calculate(). HashMap registry in the context. Median dominated by JVM start-up (~100 ms).
Strategy / Python	typing.Protocol gives a structural interface that is only checked by a type checker, not at runtime; frozen dataclass for the trip. Importing dataclasses/typing roughly doubles start-up versus the State snippet (60 vs 31 ms).
Strategy / JavaScript	Duck typing: any object with calculate(trip) is accepted; a private #strategies Map holds the registry. ES module; the role names exist only as class names.
Strategy / C++	Abstract base class with pure virtual calculate(); the context owns strategies through std::unique_ptr in a std::map, so ownership is explicit. Header-only snippet compiled with -O2; native binary starts in ~2 ms (measured inside the container).

Snippet	Observed language-specific structure
State / Java	Interface + abstract BaseState supplying refusing defaults, concrete states as nested classes; the context maps event names with a switch expression, which gives the highest function CC (6).
State / Python	Base class with refusing methods; concrete states override; the context dispatches with getattr(state, event)(), so no branch per event (CC 2). No typing/dataclass imports, hence the fastest Python start-up (31 ms).
State / JavaScript	Same shape as Python: class inheritance for default refusals and dynamic dispatch with state[event](); class fields hold the state name.
State / C++	Virtual methods returning std::unique_ptr<TripState>; [[noreturn]] refuse() in the base. The context owns the current state and replaces it on each transition; event dispatch is an if-chain (CC 6) because C++ has no reflective method lookup.
Observer / Java	Observer interface, RecordingObserver collecting events in a List and a HashMap subscriber registry; lambdas in the test runner. Static typing forces every observer to implement update().
Observer / Python	Protocol for Observer and a dict registry; broadcast is a one-line generator sum. Observers need no common base class at runtime.
Observer / JavaScript	Any object with update() can subscribe; Map registry and Array.filter for broadcast give the smallest source (24 LOC) and lowest CC (2).
Observer / C++	Hub stores std::weak_ptr so it never extends an observer's lifetime; notify() locks the pointer and skips destroyed observers - an ownership question the garbage-collected languages do not need to answer.

Technology discussion: Java and C++ express the pattern roles as nominal types checked by the compiler (interfaces, abstract classes, `override`); Python `Protocol`s and JavaScript duck typing make the same roles conventions, so the code is shorter but a missing method is found only when it runs. Dynamic dispatch by name (`getattr`, `state[event]()`) removed the event switch in Python/JS (CC 2 versus 6 in Java/C++), which matches the GoF observation that State replaces conditionals with polymorphism but the context still needs a way to map events to methods. C++ is the only version that must state ownership: the Strategy context owns strategies through `unique_ptr`, the State context replaces its state object, and the Observer hub keeps `weak_ptr`s so a subscription never keeps a disconnected observer alive – a concern garbage-collected runtimes hide. (Sources: Gamma et al. 1994; Python `typing.Protocol` PEP 544; ISO C++ smart-pointer guidelines.)

5 LLM records and human verification

Run and model: two kinds of runs. (a) Dilip Kumar Reddy used Claude Opus 5.5 in the Claude Code desktop app (paid tier, not free) on 2026-10-11 for the application study and first pattern list; the exact prompt and raw reply are in `prompts/Dilip_Kumar_Reddy_claude_*`. (b) Each member runs two prompts on a free public model (ChatGPT or Gemini, no login needed); prompt files embed the exact source excerpts given to the model. Only runs that were actually performed are listed; `data/llm.csv` is regenerated as outputs arrive.

Claims and evaluation:

Part	Model and date	Output	Evaluation
Pattern discovery (application study)	Claude Opus 5.5 (claude-opus-5-5) in Claude Code desktop app, paid tier, 2026-10-11	`prompts/Dilip_Kumar_Reddy_claude_output_01.md`	Partially correct: Repository, DI, Singleton, Adapter (Socket.IO, Stripe), Observer, Chain of Responsibility, Data Mapper and Layered Architecture were confirmed; Twilio/Cloudinary adapters are unused or out of scope; Gateway and catchAsync Decorator were missed; ON_TRIP proposal was replaced by SUSPENDED.

Generated code: the TypeScript changes and the Part 3 snippets were written with AI assistance in the same Claude Code session and are accepted only through the evidence in this report: 41 application tests, 22 fixtures per language set, `tsc` with no new errors and `-Wall -Wextra -Wpedantic` with no warnings. Each member reviews their part and signs `HUMAN_VERIFICATION.md`.

6 Findings, contributions and references

Findings: (1) The baseline already uses a clean, interface-first architecture: 16 collaborations of 10 pattern types, dominated by Repository (5) and Adapter (3) instances made possible by tsyringe DI. (2) Its variability points were conditionals: ride-type pricing and both status lifecycles had no behavioural pattern, and two defects followed from that (a completed trip could be cancelled; an admin could jump a rejected driver to approved). (3) The four requirements introduced two State, two Strategy and one Decorator instance and extended the notification Adapter/Observer, ending at 12 types and 21 instances with no removals. (4) Across languages the pattern structure stays the same; what changes is how roles are declared (nominal vs structural), how events are dispatched, and whether ownership must be explicit. Limitations: no live end-to-end run against external services; dependency drift; no upstream license; client-side observers out of scope.

Contributions (`data/contributions.csv`):

Member	Contribution and evidence
Dilip Kumar Reddy	Lead: application study, architecture, Part 1 discovery, integration, report, submission; `Part1/`, `data/patterns.csv`, `README.md`
Sai Sameer	CH01 new driver status (State) and Part 3 Observer; `Part2/changes/CH01`, `Part3/observer_ride_notification`
M Kumar	CH02 ride cancellation (State, Strategy) and Part 3 State; `Part2/changes/CH02`, `Part3/state_trip_lifecycle`
K Harish	CH03 ride types and dynamic pricing (Strategy, Decorator) and Part 3 Strategy; `Part2/changes/CH03`, `Part3/strategy_ride_fare`
S Leela Shankar	UML diagrams, Part 3 measurement harness and cross-language metrics, dataset audit, slides; `Part1/diagrams`, `Part2/diagrams`, `Part3/evidence`, `tools/run_part3.py`

References:

1. AkhilMM2000, *sarathi-backend*, `https://github.com/AkhilMM2000/sarathi-backend`, commit c8766f4.
2. E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
3. M. Fowler, *Patterns of Enterprise Application Architecture*, Addison-Wesley, 2002 (Repository, Data Mapper, Gateway).
4. M. Fowler, *Refactoring*, 2nd ed., Addison-Wesley, 2018.

5. Express 4 middleware guide, https://expressjs.com/en/guide/using-middleware.html; tsyringe, https://github.com/microsoft/tsyringe; Socket.IO v4 documentation, https://socket.io/docs/v4/.
6. PEP 544 – Protocols: structural subtyping, https://peps.python.org/pep-0544/.
7. Tools: lizard 1.17.13, PlantUML 1.2024.7, Node test runner, Docker gcc:13.

7 Team folder and datasets

```
A6/
  README.md  HUMAN_VERIFICATION.md  A6_Report.tex  A6_Report.pdf
  data/    patterns.csv, pattern_counts.csv, patterns_evolution.csv,
           cross_language.csv, llm.csv, contributions.csv
  Part1/   code/, diagrams/, evidence/, llm/
  Part2/   code/, changes/CH01..CH03/, diagrams/, evidence/, llm/
  Part3/   strategy_ride_fare/, state_trip_lifecycle/, observer_ride_notification/,
           test-cases/, evidence/, llm/, A6_slide.pdf
  prompts/ tools/
```

Screenshot of the team folder in the GitHub fork: to be added after upload.